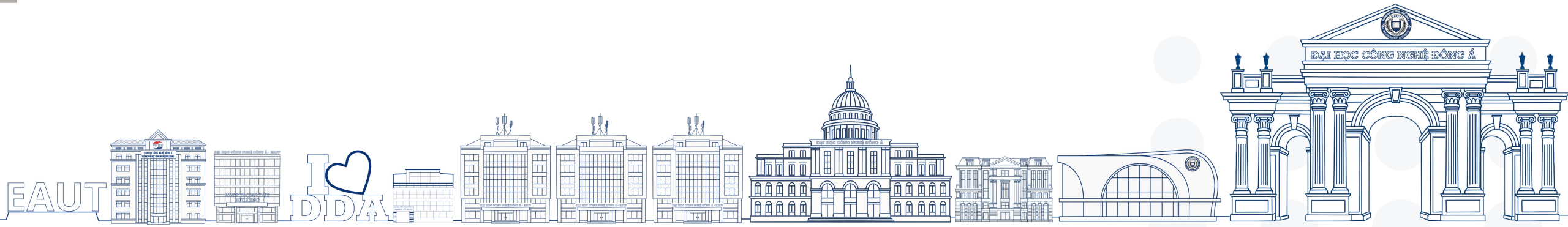
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ ĐÔNG Á
EAST ASIA UNIVERSITY OF TECHNOLOGY

Khoa Công nghệ thông tin

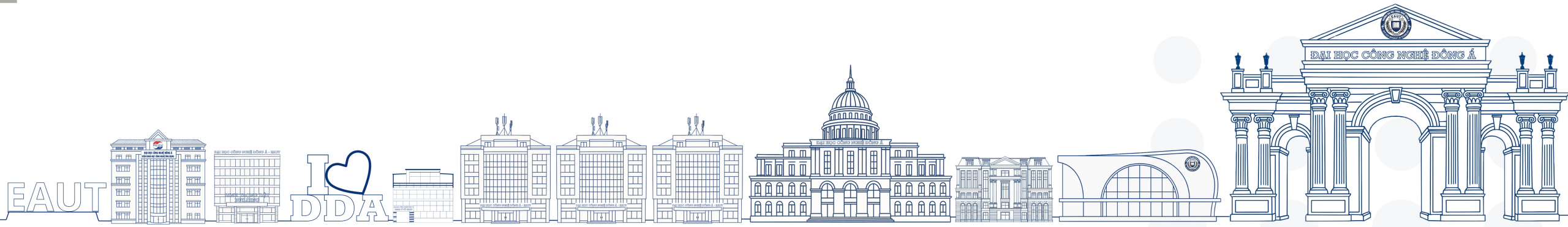
KỸ THUẬT ĐỒ HỌA MÁY TÍNH

(3TC)



TRƯỜNG ĐẠI HỌC CÔNG NGHỆ ĐÔNG Á
EAST ASIA UNIVERSITY OF TECHNOLOGY

CHƯƠNG III. CÁC GIẢI THUẬT ĐỒ HỌA CƠ SỞ



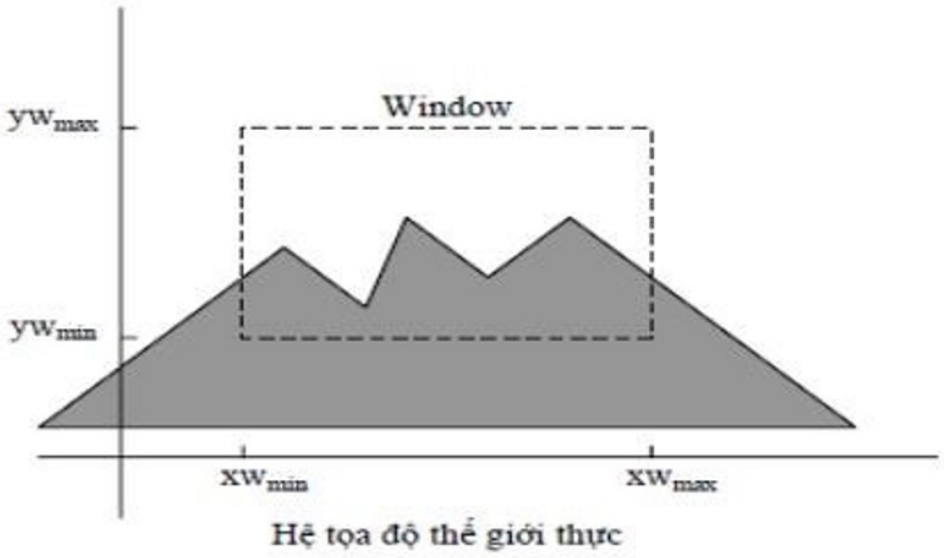
CÁC GIẢI THUẬT XÉN TỈA

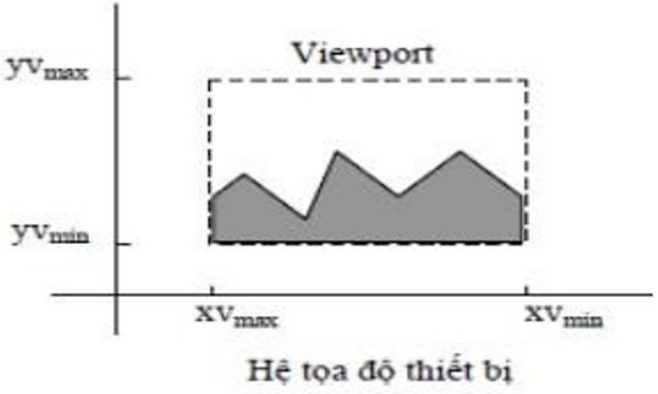
Một số khái niệm

- Mục đích của giải thuật xén tỉa (clipping) là để biểu diễn một phần hình ảnh của đối tượng

Một số khái niệm

- Cửa sổ cắt: là cửa sổ hình chữ nhật có tọa độ (x_{min}, y_{min}, x_{max}, y_{max})





Một số khái niệm

- Điểm cắt: Điểm P(x,y) nằm trong cửa sổ cắt (nhìn thấy) nếu:

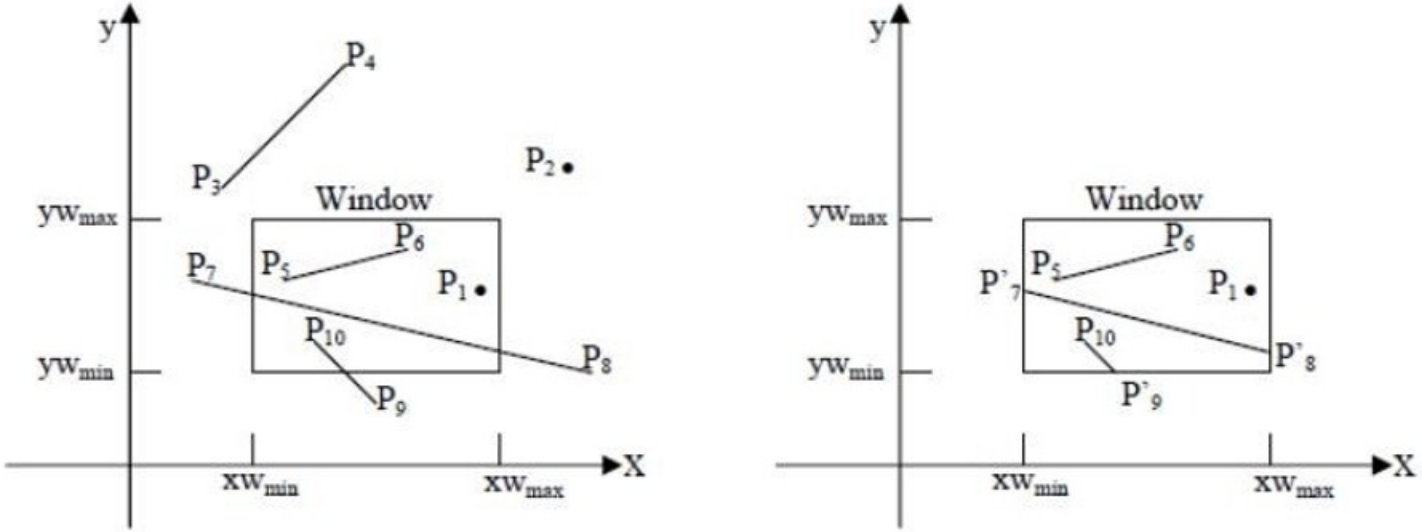
$$\begin{cases} x_{min} \leq x \leq x_{max} \\ y_{min} \leq y \leq y_{max} \end{cases}$$

- Đoạn thẳng cắt:
  - Đoạn thẳng AB được gọi là nhìn thấy nếu A,B đều nằm trong cửa sổ cắt
  - A (x1,y1), B(x2,y2) gọi là không nhìn thấy nếu:

    $x1, x2>x_{max}$; $y1,y2>y_{max}$ hoặc $x1, x2<x_{max}$; $y1,y2<y_{max}$
  - AB gọi là đoạn cắt nếu không thuộc 2 trường hợp trên

Một số khái niệm

- Đoạn thẳng cắt:
  - Ví dụ



Trước khi Clipping
(a)

Sau khi Clipping
(b)

Xác định giao hai đường thẳng

- Giao của hai đường thẳng đi qua hai điểm
  - Ví dụ đơn giản
    - Đường thẳng đi qua tọa độ (4, 2) và (2, 0) có giao với đoạn thẳng đi qua (0, 4) và (4, 0)?
  - Giải pháp
    - Xác định phương trình đường thẳng qua 2 điểm y=ax+b, trong đó a=(y2-y1)/(x2-x1)
    - từ ví dụ trên ta có: y=-2+x và y=4-x giao điểm tại (3, 1)
  - Tổng quát: nếu ta có $y=a_1+b_1x$ và $y=a_2+b_2x$ thì giao điểm sẽ ở tại: $x_i=-(a_1-a_2)/(b_1-b_2)$

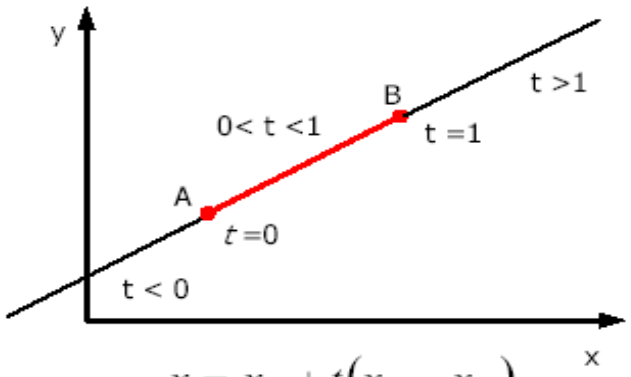
    $$y_i= a_1+b_1x_i$$
  - Các trường hợp đặc biệt: song song với trục x hay trục y, song song với nhau.

Xác định giao hai đoạn thẳng

- Nếu sử dụng phương pháp tìm giao đường thẳng: đòi hỏi kiểm tra tọa độ của giao điểm đường thẳng có nằm trong đoạn thẳng hay không?
- Phương pháp khác: biểu diễn đoạn thẳng bằng tham số
  - Đoạn thẳng 1 từ (x_A, y_A) đến (x_B, y_B)
  - Đoạn thẳng 2 từ (x_C, y_C) đến (x_D, y_D)
  - Tính toán giao của hai đoạn thẳng tại tọa độ có t, s:

$$t = \frac{(x_C - x_A)(y_C - y_D) - (x_C - x_A)(y_C - y_A)}{(x_B - x_A)(y_C - y_D) - (x_C - x_D)(y_B - y_A)}$$

$$s = \frac{(x_B - x_A)(y_C - y_A) - (x_C - x_A)(y_B - y_A)}{(x_B - x_A)(y_C - y_D) - (x_C - x_D)(y_B - y_A)}$$



$$x = x_A + t(x_B - x_A)$$
$$y = y_A + t(y_B - y_A)$$
$$x = x_C + s(x_D - x_C)$$
$$y = y_C + s(y_D - y_C)$$

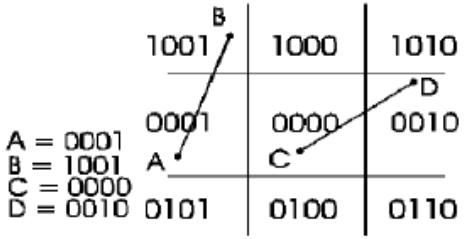
$0 \le t \le 1$ và $0 \le s \le 1$

Thuật toán cắt xén đoạn thẳng

- Các giải pháp
  - Kiểm tra từng pixel của đoạn thẳng có ở trong hcn?
  - Tính toán các điểm cắt của từng đoạn thẳng với cạnh chữ nhật cắt xén
  - Thuật toán Sutherland-Cohen: loại bỏ các đoạn không cần cắt xén bằng xét tọa độ đầu mút các đoạn thẳng -> đơn giản và hiệu quả.
  - Chia nhỏ trung điểm
  - Cyrus & Beck algorithm

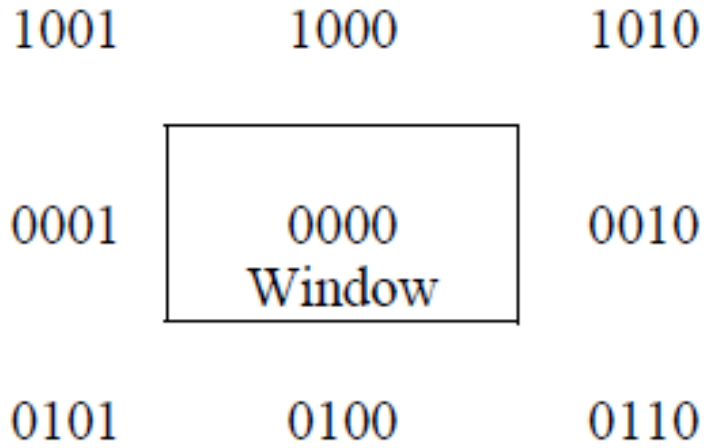
Thuật toán Sutherland-Cohen

- Mã hóa các đầu mút các đoạn thẳng
- Xác định nhanh đoạn thẳng có cần cắt xén hay không nhờ các phép toán logic AND và OR



bit 1 - left
bit 2 - right
bit 3 - below
bit 4 - above

Endpoint codes		*OR*	*AND*	*Meaning*
0000	0000	0000	0000	No clipping (1)
0001	0001	0001	**0001**	No clipping (2)
1001	0001	1001	**0001**	No clipping (3)
1001	0100	1101	0000	**Partly visible** (4)

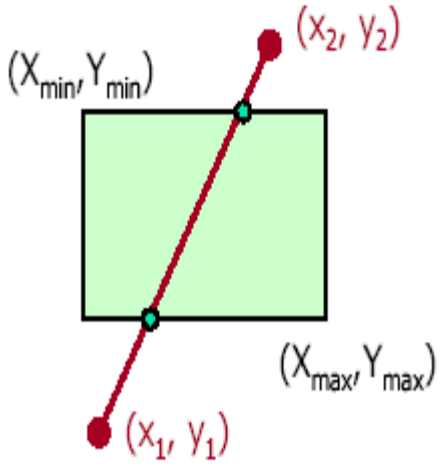


Thuật toán Sutherland-Cohen

- Kết quả phép OR hai mã đầu mút đoạn thẳng cho kết quả 0: cả hai điểm nằm trong chữ nhật.
- Kết quả phép AND hai mã đầu mút đoạn thẳng cho kết quả khác 0: cả hai điểm nằm ngoài chữ nhật
- Cắt xén
  - Giao của đoạn thẳng với các cạnh chữ nhật song song trục tung
    - x có giá trị X_{min}, X_{max} và hệ số góc $a=(y_2-y_1)/(x_2-x_1)$

      $y=y_1+a(x-x_1)$
  - Giao đoạn thẳng với các cạnh song song trục hoành
    - y có giá trị Y_{min}, Y_{max} và hệ số góc a

      $X=x_1+(y-y_1)/a$

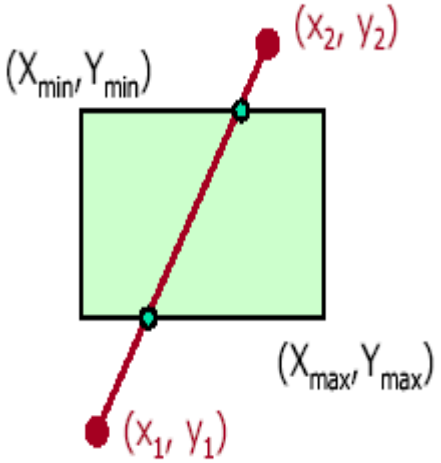


Thuật toán Sutherland-Cohen

- Ví dụ 1:

Áp dụng thuật toán cắt xén đoạn thẳng Sutherland-Cohen, hãy cắt đoạn thẳng từ *A(20,120)* đến *B(120,20)* theo cửa sổ cắt xén hình chữ nhật có giới hạn:

x_{min}=30,y_{min}=30,x_{max}=90,y_{max}=90



- Các bước thực hiện:

- Tính mã vùng cho 2 điểm A, B
- Kiểm tra đoạn thẳng nằm trong, ngoài hay cắt cửa sổ
- Tính tọa độ điểm cắt (trường hợp có cắt)
- Vẽ lại đoạn thẳng với 2 điểm cắt đã tính được

Thuật toán Sutherland-Cohen

- Ví dụ 2:

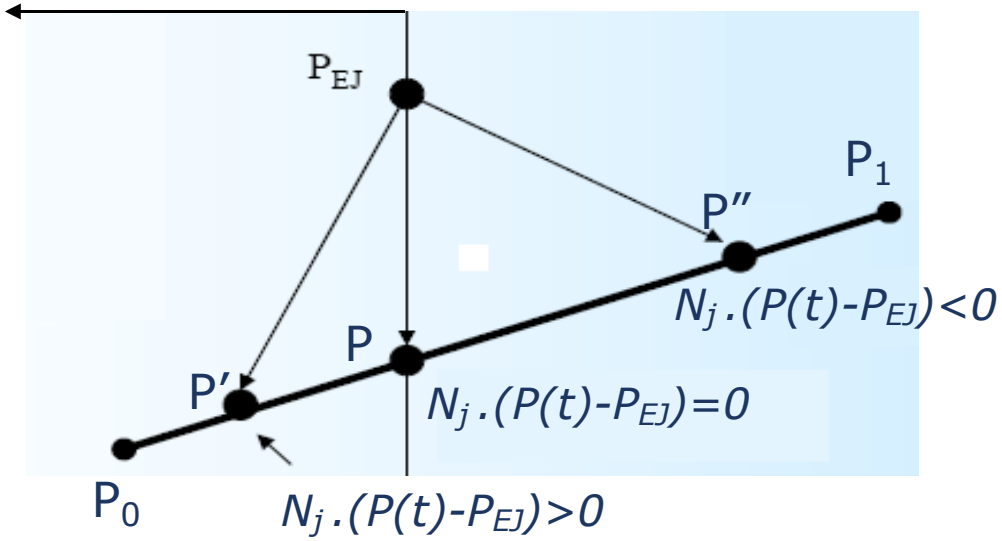
Áp dụng thuật toán cắt xén đoạn thẳng Sutherland-Cohen, hãy cắt đoạn thẳng từ *A(50,50)* đến *B(120,80)* theo cửa sổ cắt xén hình chữ nhật có giới hạn: x_{min}=30,y_{min}=30,x_{max}=100,y_{max}=100

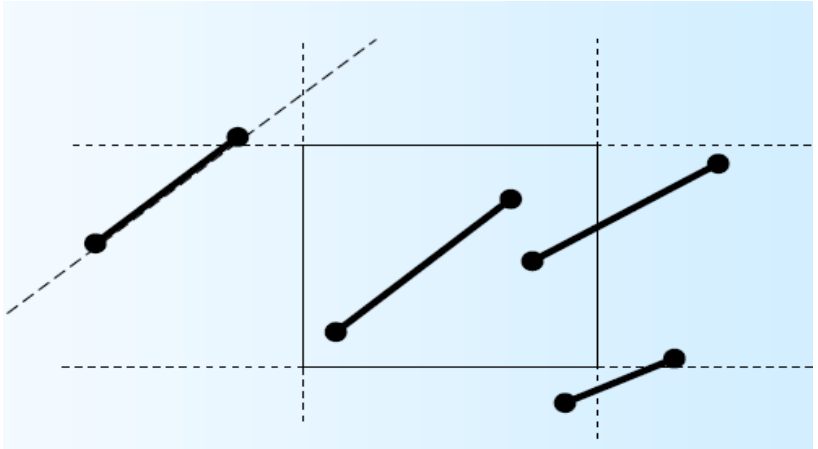
Thuật toán chia nhỏ trung điểm

- Dựa trên phương pháp chia đôi
- Đoạn thẳng được chia tại trung điểm của nó thành 2 đoạn nhỏ hơn → xác định xem thuộc loại cắt nào? → tiếp tục chia nhỏ → chỉ còn đoạn thuộc loại nhìn thấy hoặc không nhìn thấy
- Với khung nhìn trong thiết bị vật lý → đoạn thẳng thu được nhỏ hơn độ chính xác của màn hình.
- Nếu đoạn thẳng có số điểm ảnh là M → phải chia N phép chia trong đó $N = \log_2 M$.

Thuật toán Cyrus & Beck

- Sử dụng phương trình tham số:
  - $P(t)=P_0 + t(P_1-P_0)$
- Xác định giao điểm của đoạn thẳng với biên hcn, sau đó xem chúng có thuộc vùng clipping?
- Tìm một điểm trên mp clip và vecto pháp tuyến của nó





$P(t)=P_0 + (P_1-P_0)t$

$N_j .(P(t)-P_{EJ})=0 \Leftrightarrow N_j .(P_0 + (P_1-P_0)t - P_{EJ})=0$

$\Leftrightarrow N_j .(P_0 - P_{EJ}) + N_j .(P_1-P_0)t=0$

$D = P_1 - P_0$

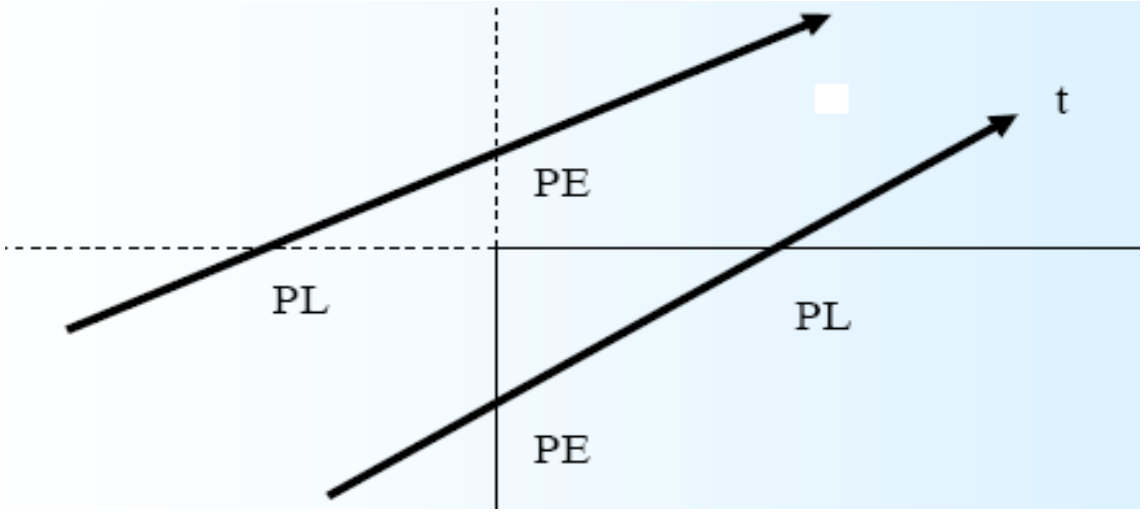
$\Rightarrow t=-N_j .(P_0- P_{EJ})/N_j .D$

$\Rightarrow$ ***Xác định hướng***

t < 0: điểm đi vào vùng clip (PE)

t > 0: đi ra vùng clip (PL)

- Sắp xếp các PE và PL lần lượt theo giá trị của t
- PL< PE : không có giao điểm
- Vẽ từ PE -> PL

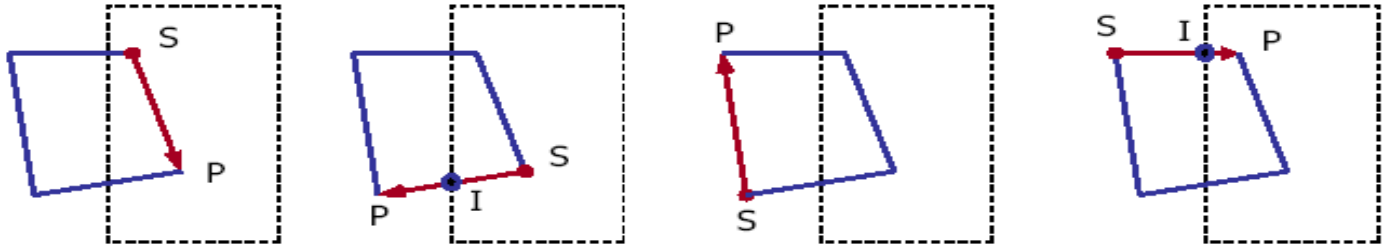


Thuật toán cắt xén vùng

- Tác giả: Sutherland-Hodgman
- Input:
  - Hình chữ nhật cắt xén
  - Vùng là đa giác được xác đinh bởi trật tự các cặp tọa độ.
- OutPut:
  - Các đa giác nằm trong cửa sổ cắt xén
- Ý tưởng thuật toán
  - So sánh lần lượt các đỉnh đa giác với biên cửa sổ
    - Đỉnh nằm ngoài, loại bỏ ngay
    - Đỉnh nằm trong, lưu trữ lại làm kết quả
    - Tính giao điểm của các cạnh đa giác vùng với cạnh chữ nhật

Thuật toán cắt xén vùng

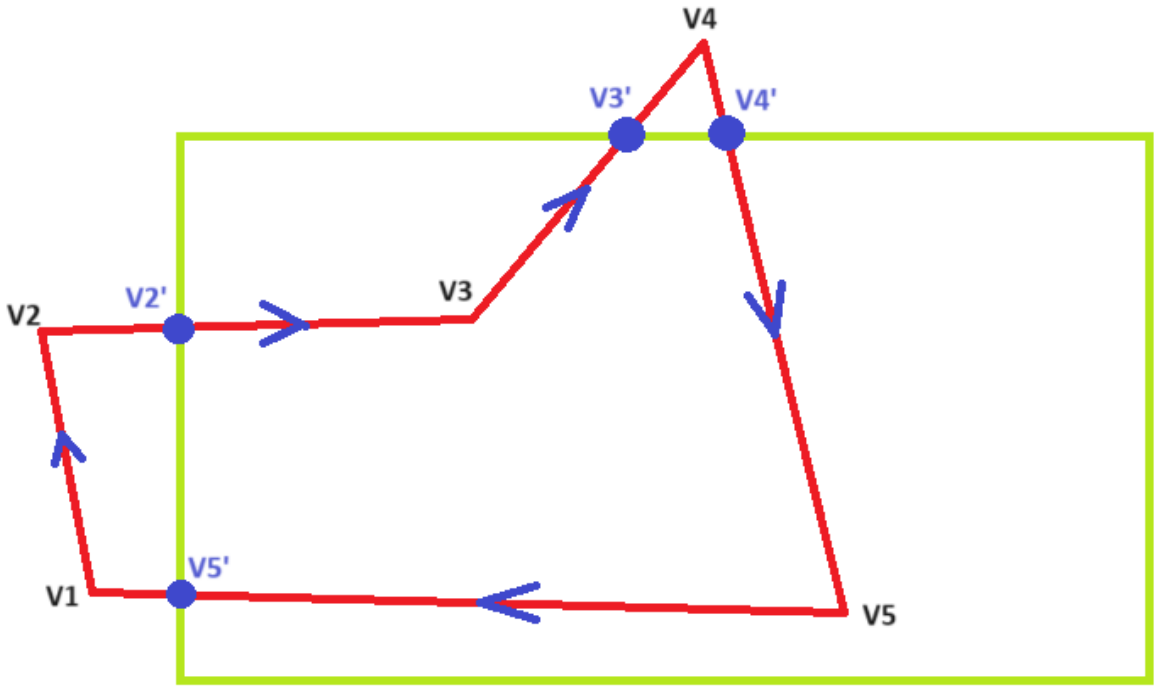
- Duyệt lần lượt (theo chiều kim đồng hồ) các cạnh đa giác:
  - Nếu đỉnh duyệt xuất phát từ trong cửa sổ theo cạnh đa giác đi ra ngoài cửa sổ: lưu trữ giao của cạnh đa giác với biên cửa sổ
  - Nếu đường đi từ ngoài vào trong cửa sổ: lưu trữ đỉnh đa giác và giao điểm
- Thí dụ xét hai đỉnh đa giác S và P:



a) Lưu P b) Lưu I c) Không lưu gì d) Lưu I, P

Thuật toán cắt xén vùng

- Ví dụ:



Xét các cạnh theo thứ tự:

V1V2: Không lưu

V2V3: Lưu V2’, V3

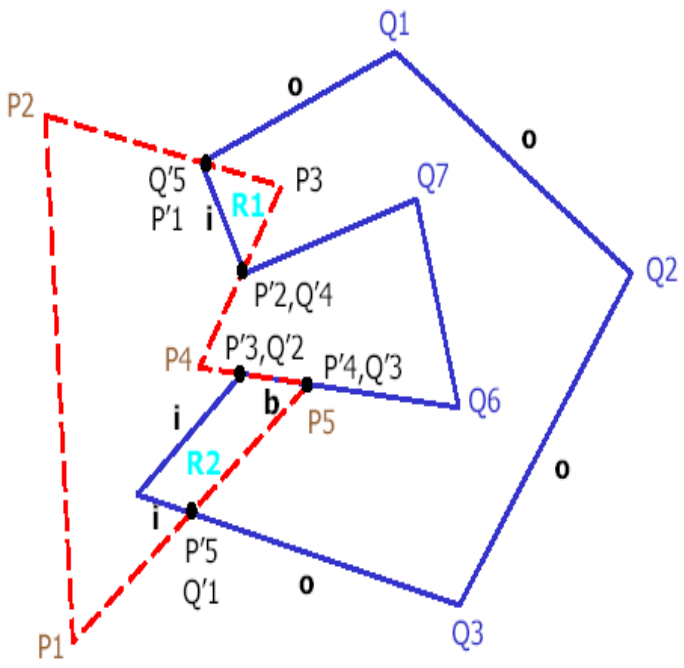
V3V4: Lưu V3’

V4V5: Lưu V4’V5

V5V1: Lưu V5’

Cắt xén vùng bằng đa giác

- Thuật toán do Clamer Schutte (Đại học Delft, Hà Lan) đề xuất
  - Cho trước hai đa giác P và Q không có lỗ hổng, không tự cắt và đỉnh của chúng được sắp xếp theo chiều kim đồng hồ
  - Hãy tìm đa giác thuộc tập P ∪ Q, P\Q, Q\P
- Các bước của thuật toán
  - Phân lớp các đỉnh của hai đa giác vào 2 danh sách: gán vào mỗi đỉnh giá trị i(inside), o(outside) hay b(boundary) phụ thuộc vào vị trí của nó so với đa giác kia
    - Pv=<(P1, o), (P2, o), (P3, i), (P4, o),(P5, b)>
    - Qv=<(Q1, o), (Q2, o), (Q3, o), (Q4, i), (Q5, b), (Q6, o), (Q7, o), (Q8, b), (Q9, b)>

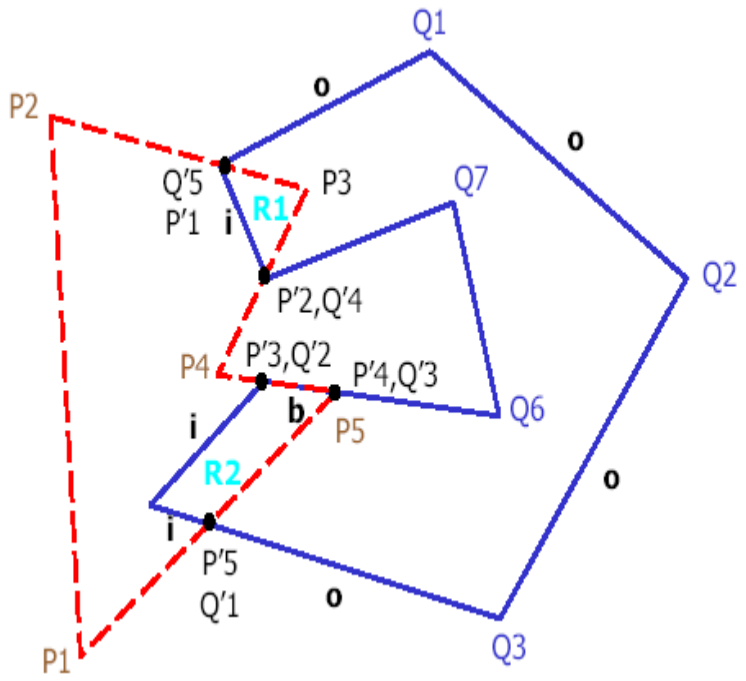


Cắt xén vùng bằng đa giác

- Tìm giao các cạnh của hai đa giác P, Q.
- Mỗi giao điểm được xen vào Pv hay Qv và đánh dấu b(boundary) để có danh sách mới

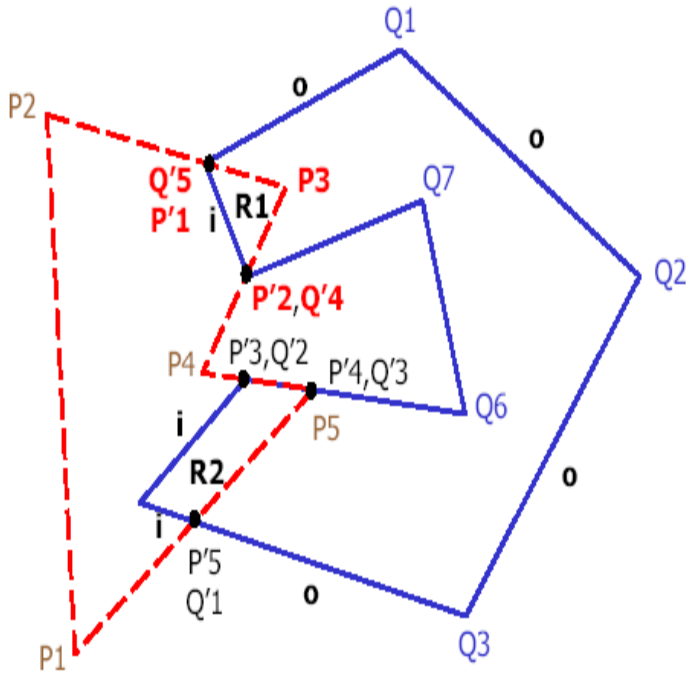
  Pef=<(P1, o), (P2, o), (P’1, b), (P3, i), (P’2, b), (P4, o), (P’3, b), (P5, b), (P’4, b), (P’5, b)>

  Qef=<(Q1, o), (Q2, o), (Q3, o), (Q’1, b), (Q4, i), (Q5, b), (Q’2, b), (Q’3, b), (Q6, o), Q7, o), (Q8, b), (Q’4, b), (Q’5, b), (Q9, b)>



Cắt xén vùng bằng đa giác

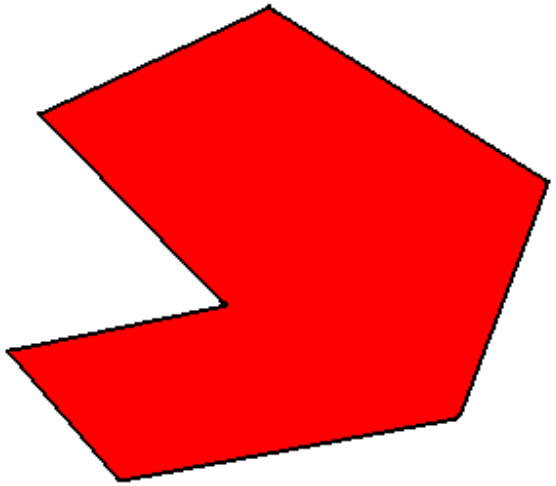
- Danh sách Pef và Qef:
  - Pef=<(P1, o), (P2, o), (P’1, b), (P3, i), (P’2, b), (P4, o), (P’3, b), (P5, b), (P’4, b), (P’5, b)>
  - Qef=<(Q1, o), (Q2,o), (Q3, o), (Q’1, b), (Q4, i), (Q5, b), (Q’2, b), (Q’3, b), (Q6, o), (Q7, o), (Q8, b), (Q’4, b), (Q’5, b), (Q9, b)>

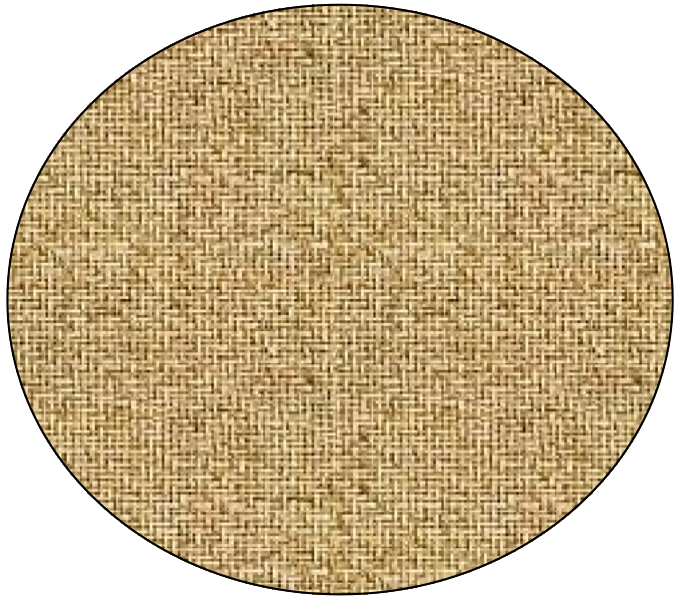


GIẢI THUẬT TÔ MIỀN KÍN

Các bước tô màu

- Xác định vị trí các điểm cần tô màu
- Quyết định tô các điểm bằng màu tương ứng



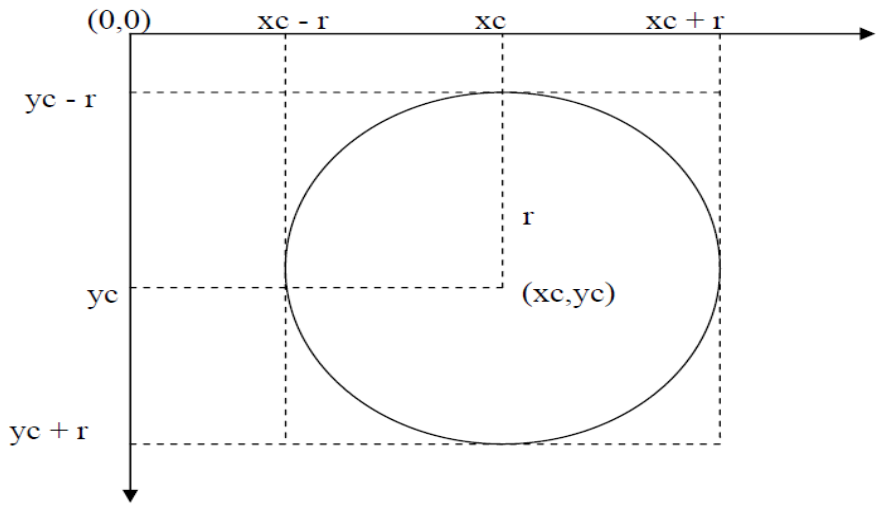


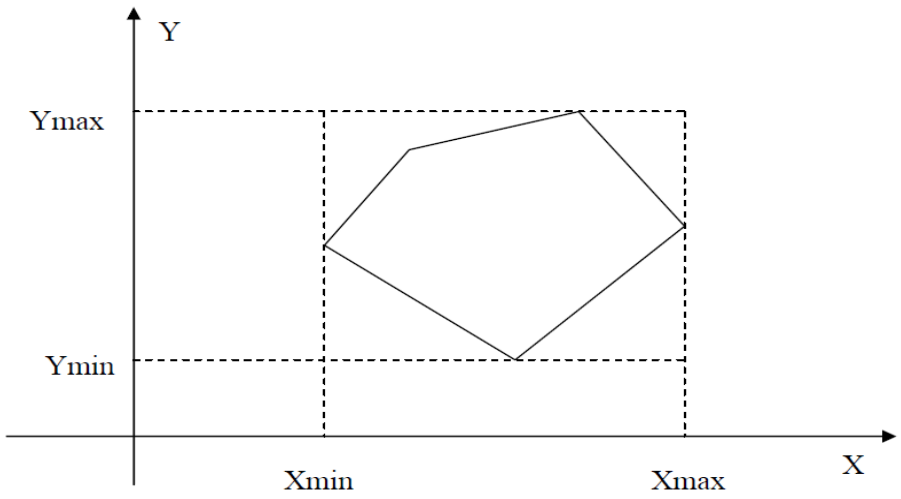
Các cách tô màu

- Tô màu theo từng điểm (tô màu đơn giản)
- Tô màu theo dòng quét
- Tô màu theo đường biên

Tô màu theo từng điểm

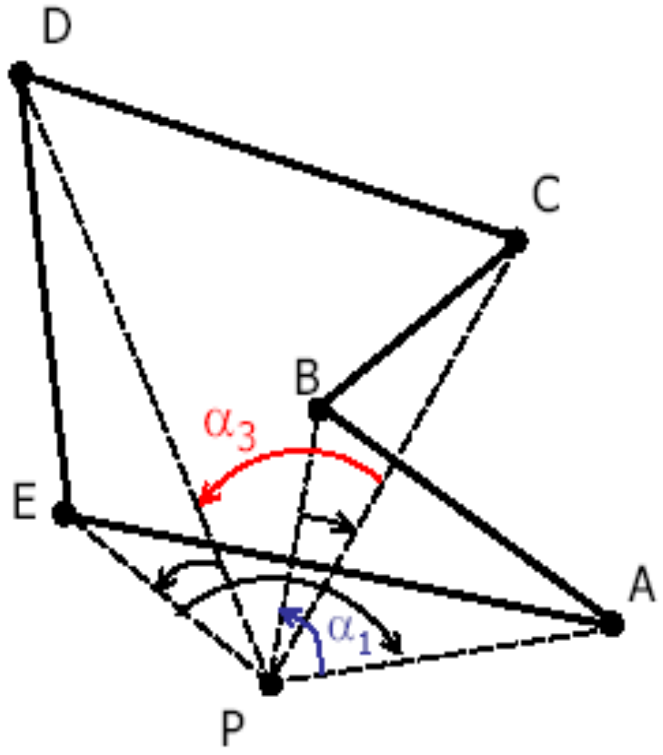
- Xác định điểm có thuộc vùng cần tô màu hay không?
- Xác lập màu sắc nếu điểm thuộc vùng cần tô màu.



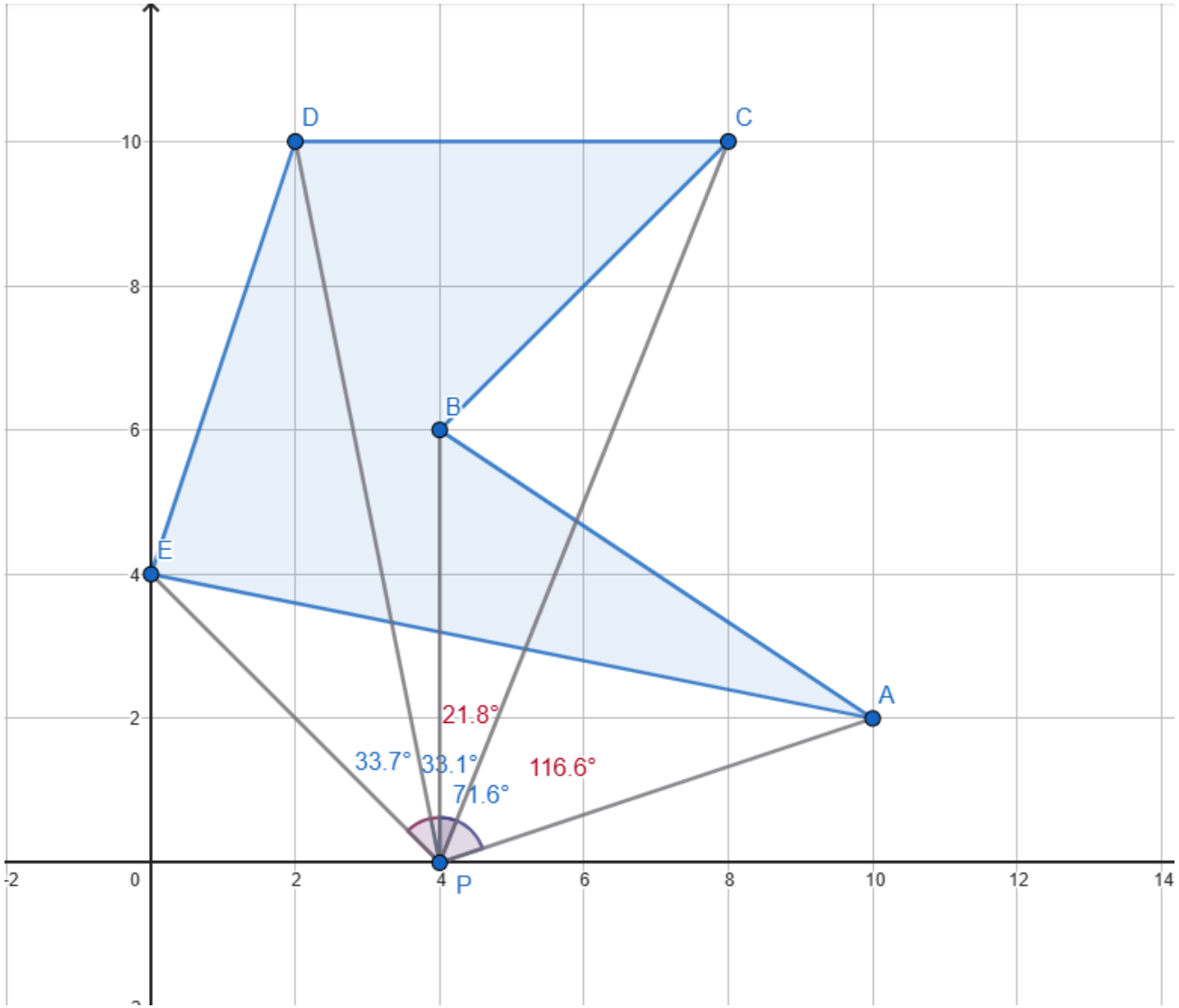


Xác định một điểm có nằm trong đa giác

- Phương pháp kiểm tra góc
  - Thí dụ, xét xem điểm P cho trước có ở trong đa giác ABCDE?
  - từ điểm P nối với các đỉnh đa giác để tạo thành các góc theo thứ tự ngược chiều kim đồng hồ
  - Các góc này có giá trị dương hoặc âm tùy theo hướng đo
  - Tính tổng các góc
    - Nếu tổng các góc bằng 0 thì P nằm ngoài đa giác
    - Nếu tổng các góc bằng 360° thì P nằm trong đa giác



Xác định một điểm có nằm trong đa giác



Xác định một điểm có nằm trong đa giác

- Cách tính số đo góc giữa 2 vector:

Ví dụ: tính số đo góc APB với A(10,2), P(4,0), B(4,6)

- Tính vector PA:

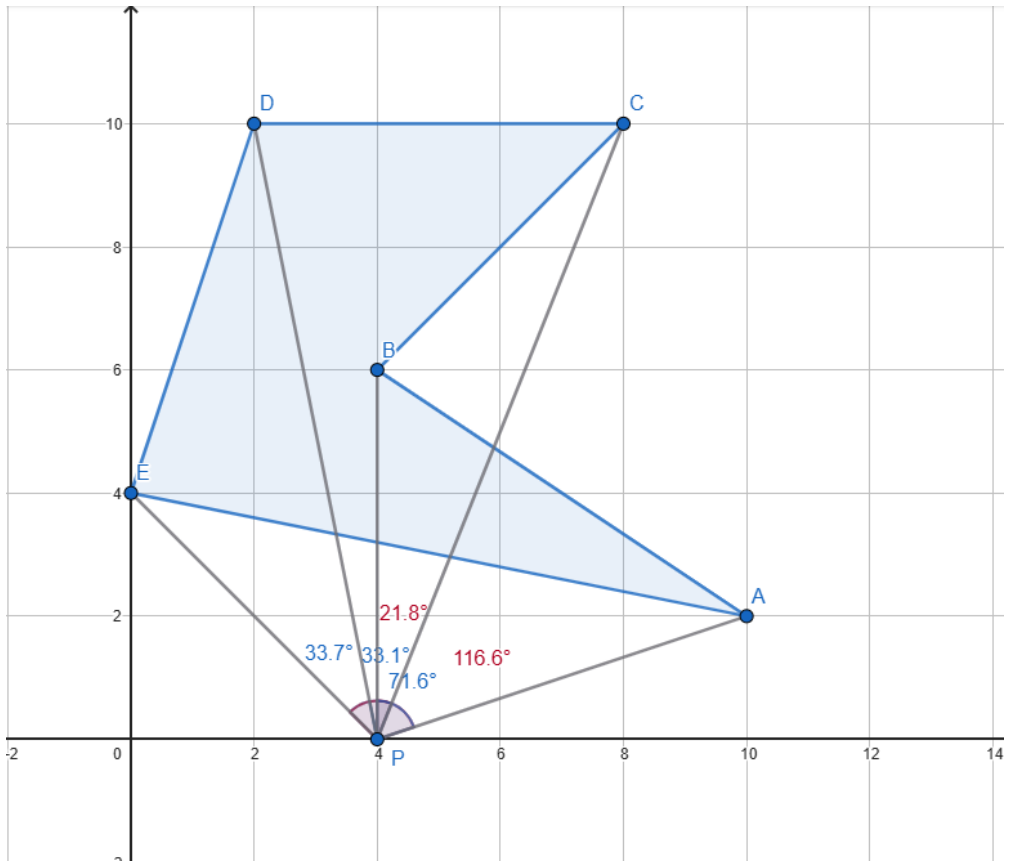
  $\overrightarrow{PA}=(10-4,2-0)=(6,2)$
- Tính vector PB:

  $\overrightarrow{PB}=(4-4,6-0)=(0,6)$
- Tính tích vô hướng:

  $\overrightarrow{PA}\times\overrightarrow{PB}=6x0+2x6=12$
- Tính tích có hướng:

  $\overrightarrow{PA}\times\overrightarrow{PB}=6\cdot6-2\cdot0=36$

Từ đó, tính góc bằng hàm atan2(36,12)=71.6

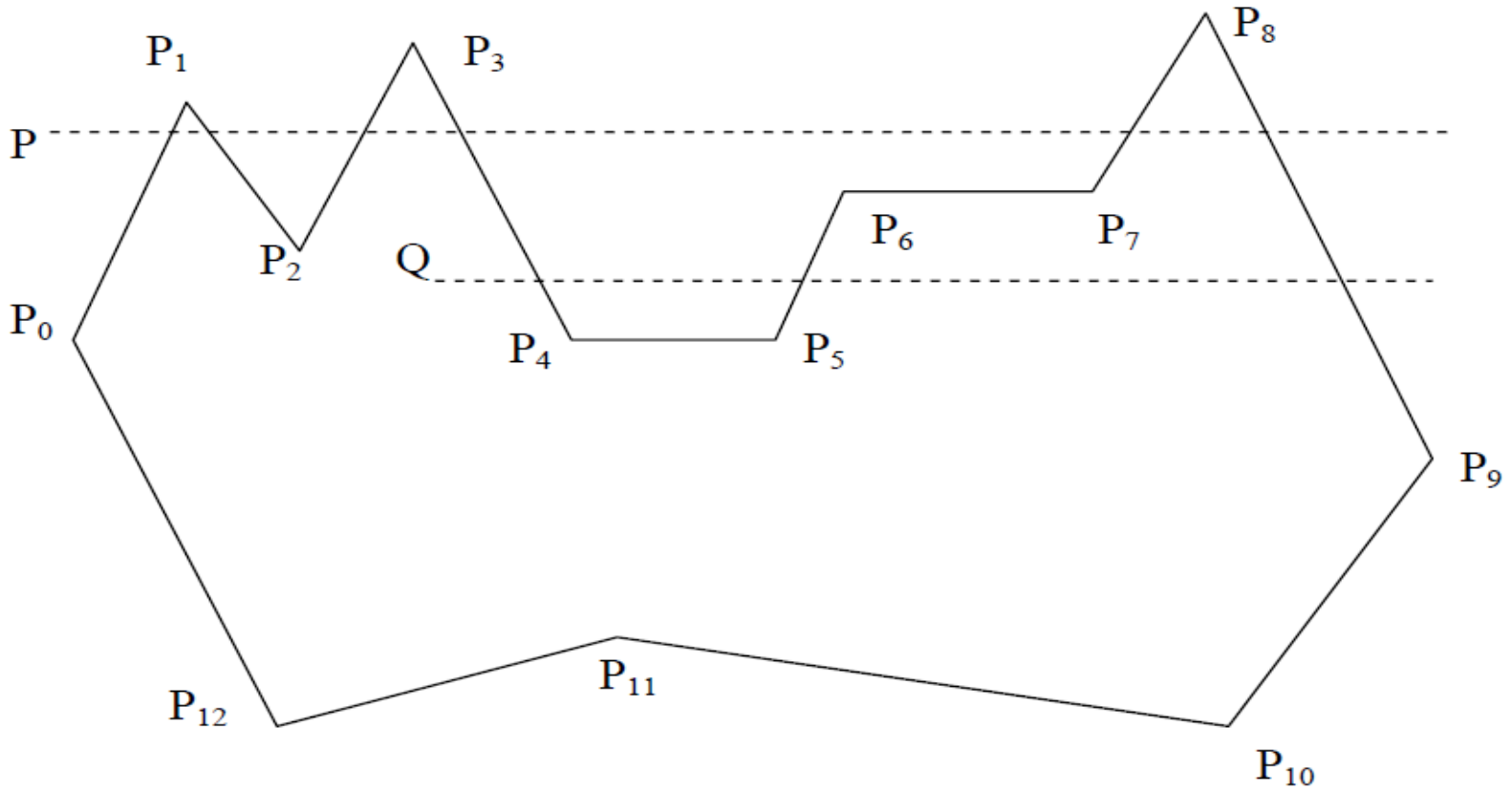


Xác định điểm trong đa giác

- Thuật toán này ứng dụng để xác định điểm độc lập hay đầu mút đoạn thẳng nằm trong đa giác
- Định lý nửa đường thẳng của Jordan
  - Từ điểm cho trước hãy vẽ tia ra cạnh tận cùng của đa giác
  - Tính tổng giao điểm của tia với các cạnh đa giác

    Nếu tổng số điểm là lẻ thì điểm đó nằm trong đa giác, ngược lại tổng số điểm là chẵn thì điểm đó nằm ngoài đa giác
  - Để dễ tính toán hãy dựng tia song song với trục tọa độ
  - Qui ước: tại các đỉnh của đa giác thì số giao điểm được tính hai lần

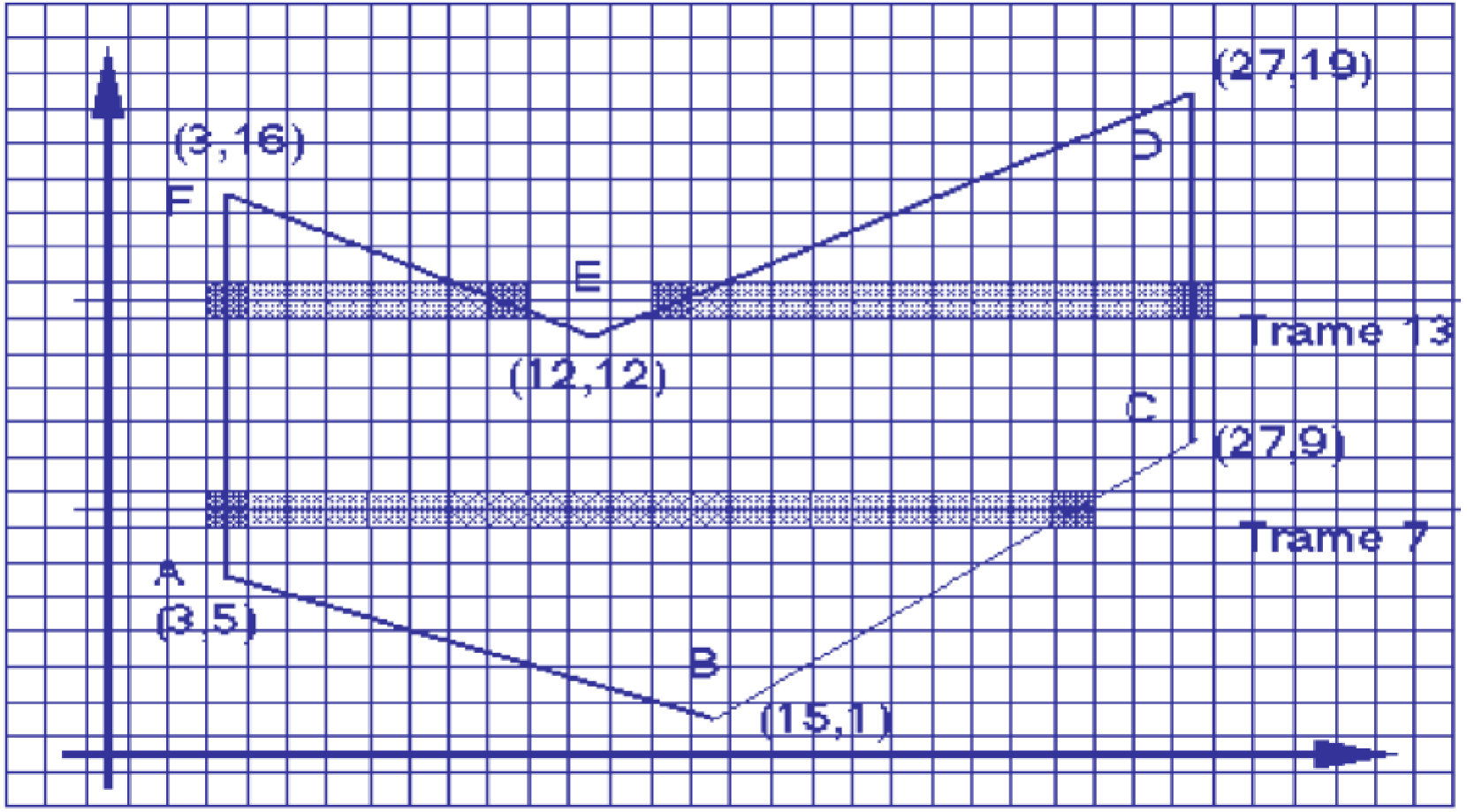
Xác định một điểm có nằm trong đa giác



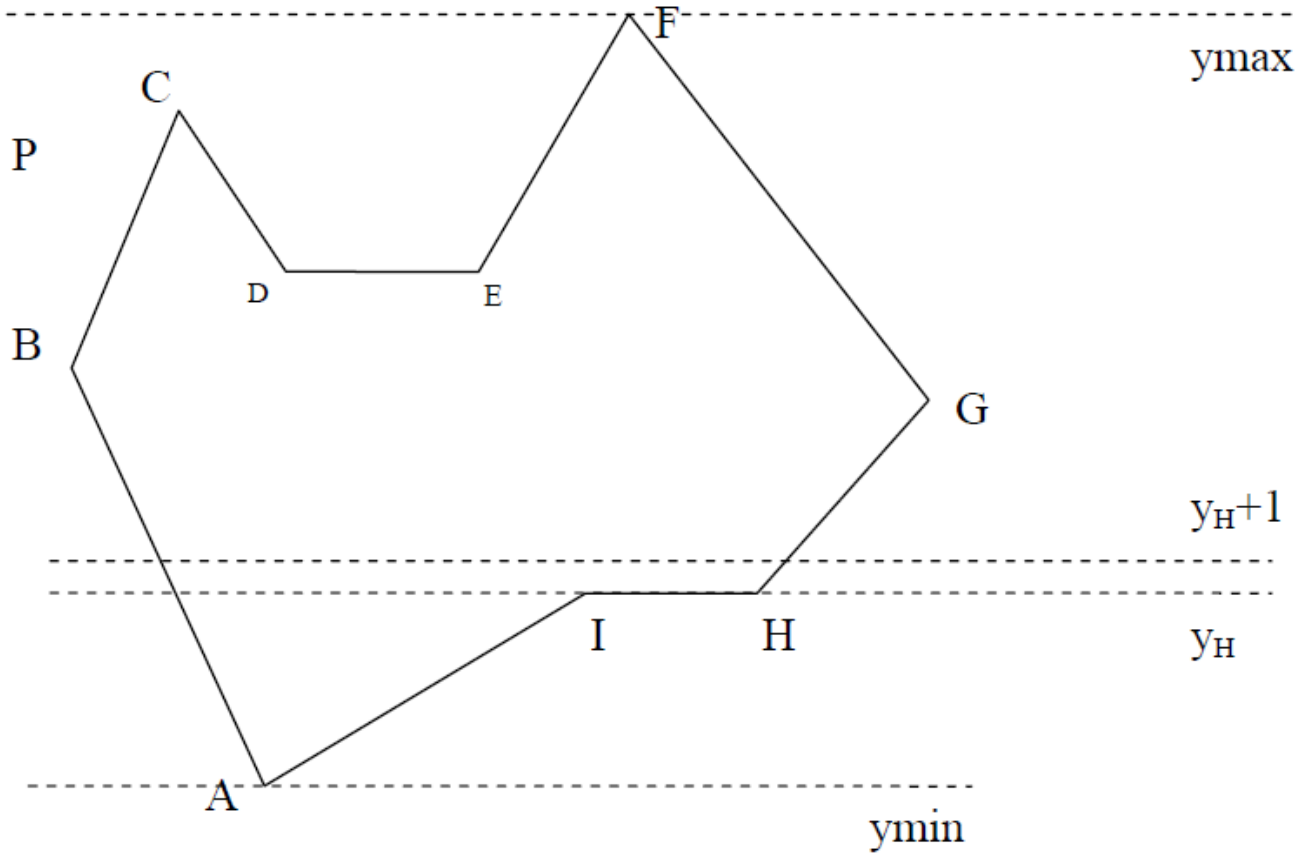
Nhận xét

- Tô màu mịn
- Có thể áp dụng cho nhiều loại đối tượng hình học khác nhau
- Tốc độ chậm

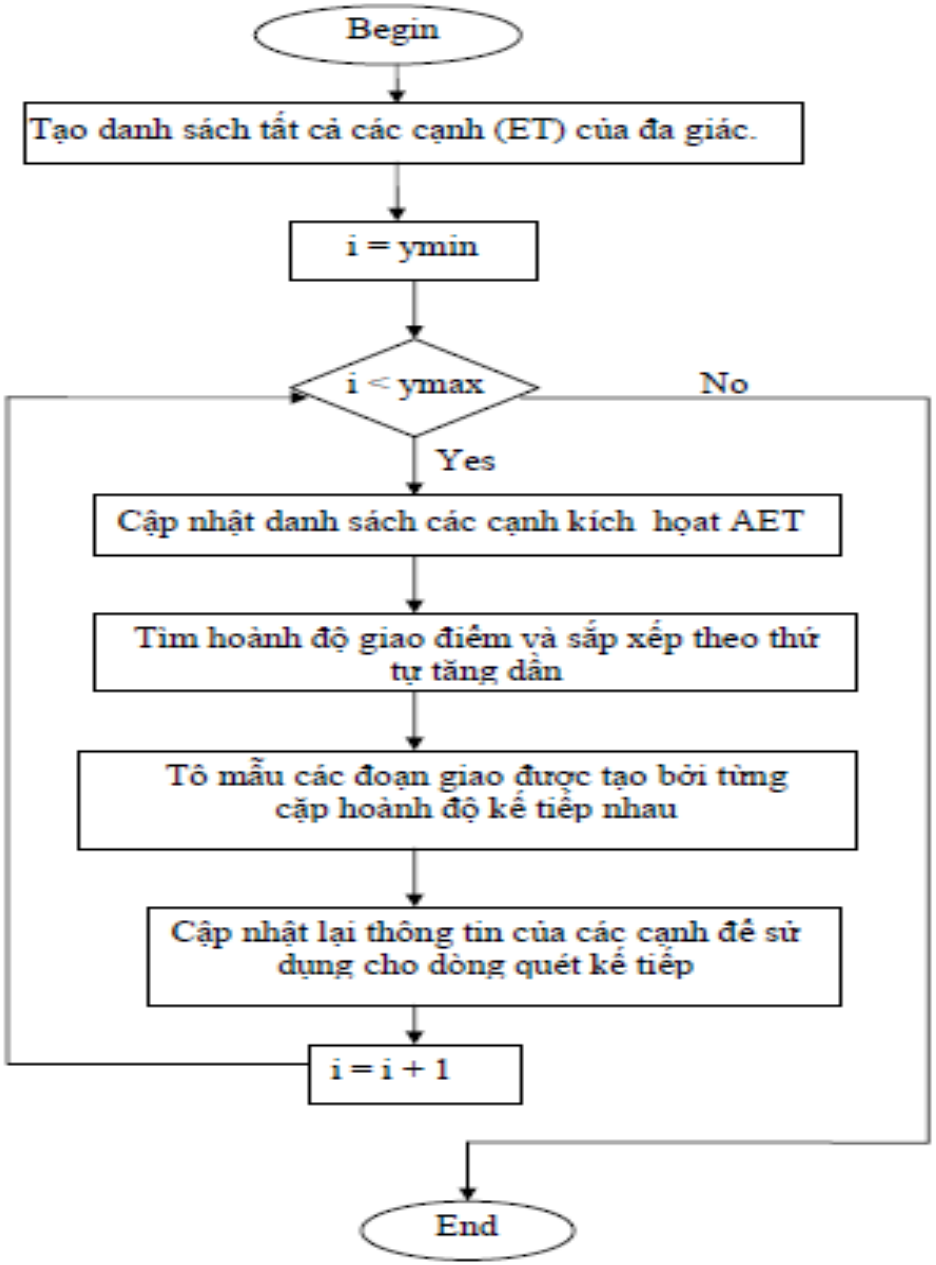
Tô màu theo dòng quét



Tô màu theo dòng quét



Thuật toán

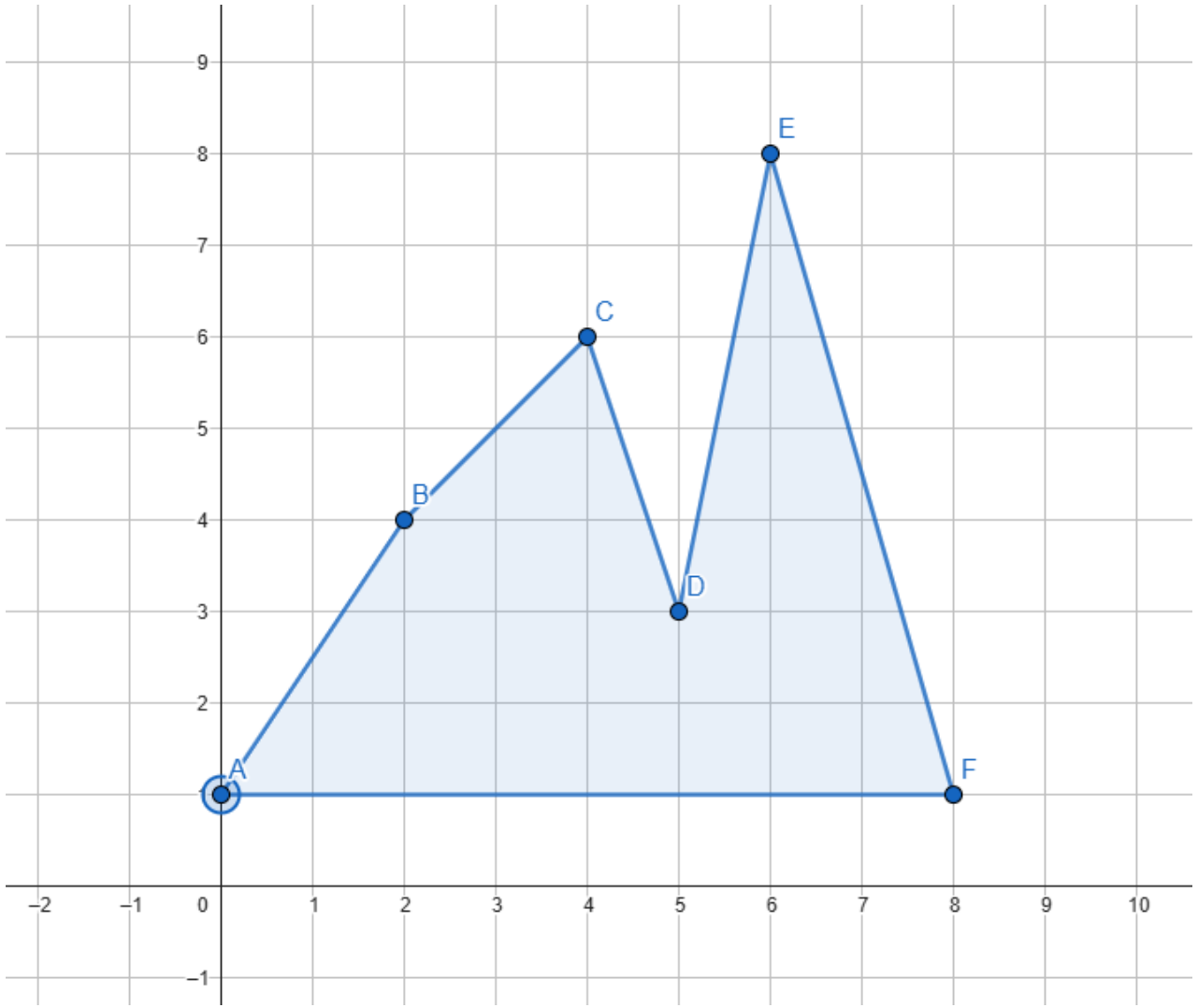


ET- Edge table
AET – Active Edge Table

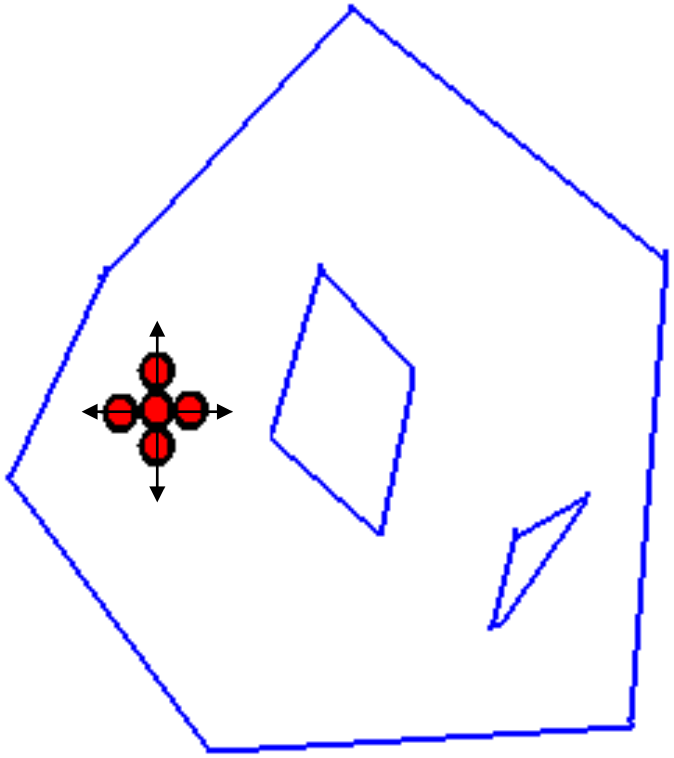
Lưu ý:

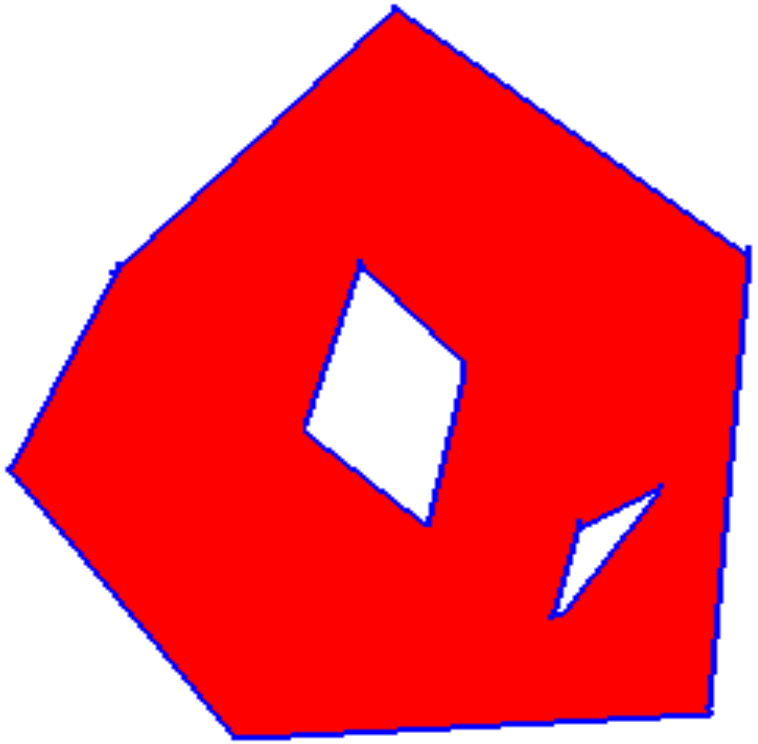
- Tập ET không chứa các cạnh song song với trục hoành
- Hoành độ giao điểm của dòng quét thứ k+1 với 1 cạnh đa giác $x_{k+1} = x_k + 1/m$ (m là hệ số góc của cạnh đó)
- Đối với 2 cạnh kề nhau (AB, BC) cùng hướng đi lên hoặc đi xuống → sẽ xét vùng quét trên AB và B'C (hướng lên trên: $y_{B'} = y_B + 1$ hoặc $y_{B'} = y_B - 1$ nếu hướng xuống dưới)
- Đối với 2 cạnh kề nhau (AB,BC) ngược hướng →xét vùng quét trên AB, BC

Thực hành tô màu theo dòng quét



Tô màu theo vùng biên





```
void floodFill4 (int x, int y, int fillColor, int oldColor)
{
  if (getPixel (x, y) == oldColor) {
    setColor (fillColor);
    setPixel (x, y);
    floodFill4 (x+1, y, fillColor, oldColor);
    floodFill4 (x-1, y, fillColor, oldColor);
    floodFill4 (x, y+1, fillColor, oldColor);
    floodFill4 (x, y-1, fillColor, oldColor);
  }
}
```